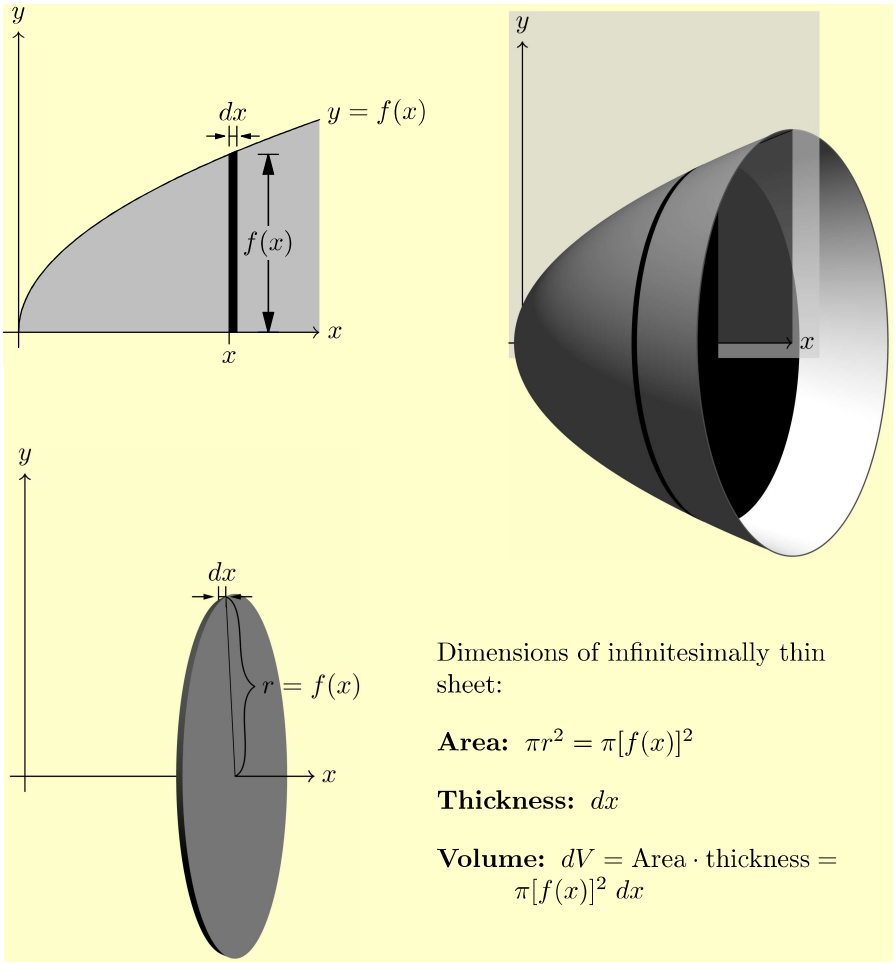
An Asymptote tutorial

Charles Staats III

May 12, 2022

Contents



1 First Steps

1.1 Introduction

Janet is a calculus teacher. She is currently teaching her students how to find volumes of solids of revolution via the "disk method." She would like to produce a diagram to illustrate the method—something like the diagram shown on page 3.

As an experienced user of TikZ, Janet does not think she would have trouble producing the diagram in the top left of the figure. She also believes she could

get the diagram in the bottom left with some fiddling. However, she simply does not believe the three-dimensional capabilities of TikZ are up to drawing a diagram like that in the top right—at least, not without far more time and fiddling than Janet is willing to put in for a single diagram.

Janet's husband Vincent is a programmer. He has some familiarity with the programming language Asymptote, which is especially designed to produce vector graphics and has some fairly substantial three-dimensional capabilities. Working with her husband, Janet decides to try to draw the figure using Asymptote.

1.2 Hello World

Janet already has an up-to-date installation of TeXLive. On the off-chance that this includes an Asymptote installation, she attempts a simple Hello World program. She uses her favorite text editor[1] to produce a document consisting of the single line

```
label("Hello world!");
```

and saves it as `hello_world.asy`. She then goes to the command line and types `asy hello_world`. The result is an eps file named `hello_world.eps`. Opening it, she sees a single page with the following printed on it:[2]

Hello world!

Thus, to the surprise of both Janet and her husband, it appears that Asymptote is already installed on her computer.[3]

Since Janet uses `pdflatex`, she finds it annoying to import eps files, and would prefer that the "graphic" be output in another format. Adding one line to her Asymptote file causes it to output a pdf file instead:

```
settings.outformat = "pdf";
label("Hello world!");
```

Vincent notes that every line should end with a semicolon. Since TikZ behaves the same way, Janet does not find this too difficult to remember.

Now that Janet has a pdf file containing her "graphic," she decides to import it into a latex file. Having done so, she is pleased to notice that Hello World is printed in the same font as the rest of her document (Computer Modern). However, she considers it unfortunate that the font size in the "graphic" is larger than in the rest of her document. Vincent asks her what size she would like

[1] She is inclined to use TeXShop, since she already has it, but Aquamacs would really be more suitable. Vincent, who is a die-hard fan of the command line, recommends using the command-line editor `nano`. On a Windows machine, the simplest thing to use would be Notepad.

[2] Not including the yellow background, of course.

[3] This was more or less my experience, working on Mac OS X. If you are not fortunate enough to have had this miracle happen to you, see Appendix B and the installation instructions in the Asymptote manual.

the font in her Asymptote graphics. Being told that the desired font size is 10 points, he proposes the following:

```
settings.outformat = "pdf";
defaultpen(fontsize(10pt));
label("Hello world!");
```

The result is

Hello world!

which looks nicer with the rest of the document.

1.3 Interpreting the documentation

The `label()` command used in the Asymptote code above is described in Section 4.4 of the Asymptote manual, with the following declaration:

```
void label(picture pic=currentpicture, Label L, pair position,
           align align=NoAlign, pen p=currentpen,
           filltype filltype=NoFill)
```

Janet finds this a bit overwhelming. Vincent suggests that she ignore everything with an `=` sign in it, since those are all optional arguments, and think of such a declaration simply as

```
label(Label L, pair position);
```

Essentially, if Janet wants to use this command to add a label to her picture, she should tell what Label to add and where to add it. For instance, the line

```
label("Hello world", (0,0));
```

would add the text "Hello world" to the picture at position $(0, 0)$. Given this description, Janet wonders why the program she already wrote worked; shouldn't she have been required to specify a position? Vincent confirms that according to the documentation, the line `label("Hello world");` without any position should not have worked. He admits that the documentation is sometimes not completely accurate, which Janet does not find encouraging.

It seems rather peculiar to Janet that the equals sign should be what indicates that an argument can be ignored. Vincent explains that the equals sign provides a default value; when there is a default value, the program knows what to do if the user does not specify a value.

Here are the optional arguments for the `label` command:

type	name	default value
`picture`	`pic`	`currentpicture`
`align`	`align`	`NoAlign`
`pen`	`p`	`currentpen`
`filltype`	`filltype`	`NoFill`

These optional arguments behave something like key-value assignments. For instance, if Janet had wanted to change the text size of just the single line, rather than the entire picture, she could have set the key `p` to value `fontsize(10pt)` as follows:

```
settings.outformat = "pdf";
label("Hello world", p = fontsize(10pt));
```

Note that `fontsize(10pt)` is an object of type `pen`, just as `"Hello world"` (including the quotation marks) is an object of type `Label`.[4] The output is the same as before, since the picture has only one piece of text:

Hello world

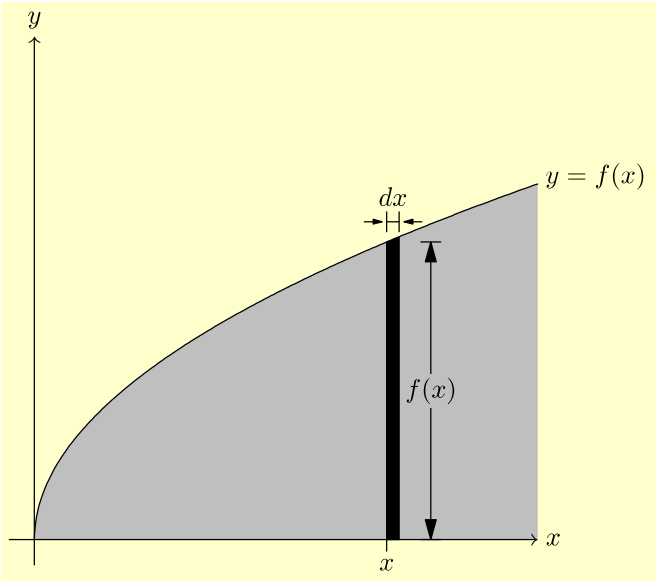
Janet asks how to define a single key that can set several others, as in TikZ styles. Vincent says that this is not possible in Asymptote, although he can see how it might be useful.

2 Drawing a two-dimensional image

2.1 Lines and sizing

Having determined that Asymptote is already installed on her computer, Janet decides to use it to draw a picture of the two-dimensional region that will be revolved. She could do this using TikZ, but Vincent recommends that she get some basic practice drawing with Asymptote before tackling a three-dimensional picture. Here is, roughly, what Janet would like:

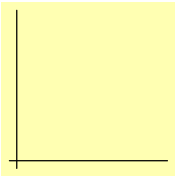
[4] Technically, `"Hello world"` is of type `string`, but strings can be treated as Labels in Asymptote.



First of all, she tries drawing the x-axis as a line from $(-0.1, 0)$ to $(2, 0)$, and the y-axis as a line from $(0, -.1)$ to $(0, 2)$;

```
settings.outformat="pdf";
draw((-.1,0) -- (2,0));
draw((0,-.1) -- (0,2));
```

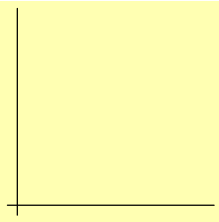
The result is a mark that is barely visible because it is so short. Vincent lets Janet know this is because, by default, Asymptote interprets one unit to mean one point—roughly 0.035 centimeters. Janet thinks that the TikZ default of 1 centimeter is much more reasonable. This can be arranged by adding the line `unitsize(1cm);` to the code:



```
settings.outformat="pdf";
unitsize(1cm);
draw((-.1,0) -- (2,0));
draw((0,-.1) -- (0,2));
```

The final product should be larger still, but is kept this size for now to save space.

There's one more kind of sizing option for Asymptote: the `size` command. In its simplest usage, this command takes a single length for an argument—in the code below, `3cm`—and makes the final picture as large as possible, keeping the same height-to-width ratio, such that neither the width nor the height exceeds the specified dimension (3 centimeters).



```
settings.outformat="pdf";
size(3cm);
draw((-.1,0) -- (2,0));
draw((0,-.1) -- (0,2));
```

The `size` command has several important variations:

- `size(real, real)` takes two dimensions—a maximum width and a maximum height, in that order. Thus, for instance, code that begins with the command `size(2cm, 3cm);` will ordinarily produce a picture that is either 2 centimeters wide (and $\leq$ 3 centimeters tall) or a picture that is 3 centimeters tall (and $\leq$ 2 centimeters wide). In either case, the height-to-width ratio is preserved.
- If either argument in `size(real, real)` is zero, it is ignored. Thus, for instance, a picture that includes the command `size(4cm, 0);` will ordinarily be scaled so that the width is exactly 4 centimeters. The height will be the "natural" height for this scaling factor.
- The command `size(real, real, keepAspect=false);` will scale the width and height independently so that the resulting picture has exactly the specified width and height, but the height-to-width ratio is allowed to change. Thus, for instance, if a circle is drawn on a picture that has this type of size command, the circle is likely to end up looking like an ellipse.

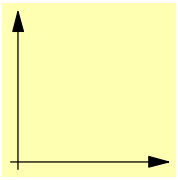
One of Janet's frustrations with TikZ has been that it is difficult to produce a picture that has exactly the desired width (or height). She is gratified to learn that this will be much easier with Asymptote.

2.2 Arrowheads

With the axis lines drawn, Janet thinks that they should have arrows indicating the directions. Vincent agrees that the axes should have arrows. Janet's students do not care about arrows.

Arrows can be added on the end of the line by using the optional parameter `arrow` in the draw command:

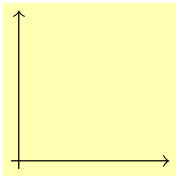
```
settings.outformat="pdf";
unitsize(1cm);
draw((-.1,0) -- (2,0), arrow=Arrow);
draw((0,-.1) -- (0,2), arrow = Arrow);
```



Vincent thinks this looks fairly nice. Janet wants to imitate the TEX-style arrowheads $\rightarrow$, as she is used to being done in TikZ. Looking in the Asymptote manual, she and Vincent find the following styles for arrowheads:

Asymptote code	appearance
`draw((0,0)--(1,0),arrow=Arrow());`	
`draw((0,0)--(1,0),arrow=ArcArrow());`	
`draw((0,0)--(1,0),arrow=Arrow(SimpleHead));`	
`draw((0,0)--(1,0),arrow=ArcArrow(SimpleHead));`	
`draw((0,0)--(1,0),arrow=Arrow(HookHead));`	
`draw((0,0)--(1,0),arrow=ArcArrow(HookHead));`	
`draw((0,0)--(1,0),arrow=Arrow(TeXHead));`	

The last, the `TeXHead` style, is more what Janet has in mind:

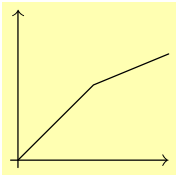


```
settings.outformat="pdf";
unitsize(1cm);
draw((-.1,0) -- (2,0), arrow=Arrow(TeXHead));
draw((0,-.1) -- (0,2), arrow = Arrow(TeXHead));
```

2.3 Curved paths

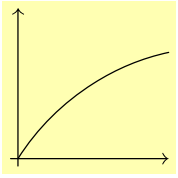
Next, Janet would like to draw the parabola function, which is supposed to look something like the graph of $y = \sqrt{x}$. Here's a first attempt, with a path through the three points $(0, 0)$, $(1, 1)$, and $(2, \sqrt{2})$:

```
settings.outformat="pdf";
unitsize(1cm);
draw((-.1,0) -- (2,0),
    arrow=Arrow(TeXHead));
draw((0,-.1) -- (0,2), arrow =
    Arrow(TeXHead));
draw((0,0) -- (1,1) -- (2,sqrt(2)));
```



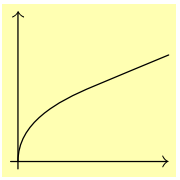
This does not look very nice at all. Substituting `..` for the connector `--` can at least make the path look smooth:

```
settings.outformat="pdf";
unitsize(1cm);
draw((-.1,0) -- (2,0),
    arrow=Arrow(TeXHead));
draw((0,-.1) -- (0,2), arrow =
    Arrow(TeXHead));
draw((0,0) .. (1,1) .. (2,sqrt(2)));
```



While this is a significant improvement, Janet would also like to make the tangent at the origin vertical. This can also be specified:

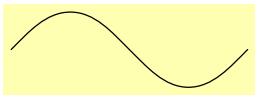
```
settings.outformat="pdf";
unitsize(1cm);
draw((-.1,0) -- (2,0),
    arrow=Arrow(TeXHead));
draw((0,-.1) -- (0,2), arrow =
    Arrow(TeXHead));
draw((0,0){up} .. (1,1) ..
    (2,sqrt(2)));
```



This looks more or less like what Janet had in mind.

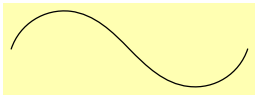
Note that `up` is really just short for `(0,1)`. If Janet wanted a direction other than `up` (or `down`, `right`, or `left`, which are similar), she could specify the tangent direction explicitly as an ordered pair. For instance, here is a rough approximation of a sine curve:

```
settings.outformat = "pdf";
unitsize(0.5cm);
draw((0,0){(1,1)} .. {right}(pi/2,1)
      .. {(1,-1)}(pi,0) ..
    {right}(3*pi/2,-1)
      .. {(1,1)}(2*pi, 0));
```



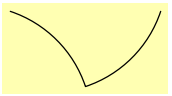
Without the tangent directions specified, Asymptote does a very nice job of connecting the dots to form a smooth curve, but it doesn't really look like a sine curve. The ends in particular are much too steep:

```
settings.outformat = "pdf";
unitsize(0.5cm);
draw((0,0) .. (pi/2,1) .. (pi,0)
      .. (3*pi/2,-1) .. (2*pi, 0));
```



Except at the first and last points, the tangent direction at a point can be specified either before or after the point—or, if a corner is desired, at both:

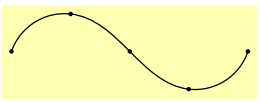
```
settings.outformat = "pdf";
unitsize(0.5cm);
draw((0,2) .. {(1,-3)}(2,0){(1,1/3)}
    .. (4,2));
```



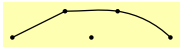
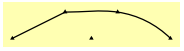
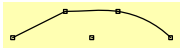
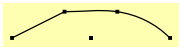
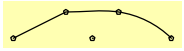
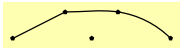
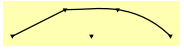
2.4 Markers on paths

If Janet wants to see what the points on a path were actually specified, she can use the `marker` option to the `draw` command:

```
settings.outformat="pdf";
unitsize(0.5cm);
draw((0,0) .. (pi/2,1) .. (pi,0)
      .. (3*pi/2,-1) .. (2*pi, 0),
      marker=MarkFill[0]);
```



Designing your own markers is not that difficult, but requires more knowledge than is currently available. Here are the built-in markers:

built-in option	description	appearance
`Mark[0]`	open circle	
`MarkFill[0]`	filled circle	
`Mark[1]`	open triangle	
`MarkFill[1]`	filled triangle	
`Mark[2]`	open square	
`MarkFill[2]`	filled square	
`Mark[3]`	open pentagon	
`MarkFill[3]`	filled pentagon	
`Mark[4]`	open triangle (upside down)	
`MarkFill[4]`	filled triangle (upside down)	
`Mark[5]`	x-mark	
`Mark[6]`	asterisk	

2.5 Circles and ellipses

The path

```
unitcircle
```

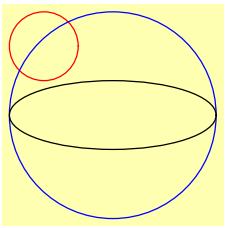
is a unit circle. The function

```
path circle(pair c, real r);
```

returns a circle centered at `c` with radius `r`. The function

```
path ellipse(pair c, real a, real b);
```

produces an ellipse centered at `c` with horizontal diameter `2a` and vertical diameter `2b`.



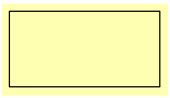
```
settings.outformat="pdf";
size(3cm);
draw(circle((0,1), 0.5), red);
draw(circle((1,0), 1.5), blue);
draw(ellipse((1,0), 1.5, 0.5));
```

2.6 Boxes and polygons

The function

```
path box(pair a, pair b);
```

returns a cyclic path that is a rectangle of which `a` and `b` are opposite corners:

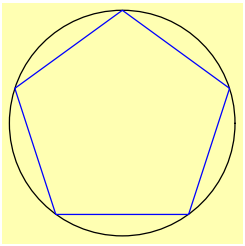


```
settings.outformat="pdf";
unitsize(1cm);
draw(box((0,0), (2,1)));
```

The function

```
path polygon(int n);
```

returns a cyclic path that is a regular polygon with `n` sides, all of whose corners lie on the unit circle:

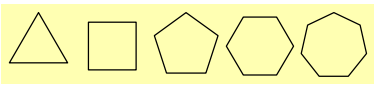


```
settings.outformat="pdf";
unitsize(1.5cm);
draw(unitcircle);
draw(polygon(5), blue);
```

2.7 Transformations: shifting, scaling, rotating, etc.

Janet observes that the `polygon` function above seems to be quite limited. What happens if she wants to draw a polygon at a different position, or a different size? Or upside down? Obviously, she could simply construct the path directly, but she thinks that the `polygon` function ought to allow for more flexibility.

After consulting the documentation, Vincent realizes that such flexibility is not necessary: Janet can still change the path after it has been created, but before it is drawn, using the `transform` type. Here's an example of using a `shift` transform to draw several polygons side by side:

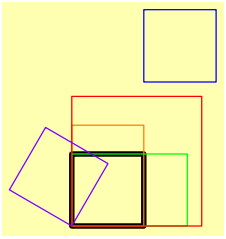


```
settings.outformat="pdf";
size(5cm);
for (int n = 3; n <= 7; ++n) {
    draw(shift(2.2*n, 0) *
    polygon(n));
}
```

Note also the use of a `for` loop to repeat the same code multiple times.

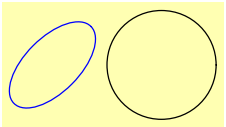
Warning: Since Vincent is an experienced programmer in C-like languages, his first instinct is to use `n++` instead of `++n` to increment `n`. Unfortunately, this *does not work* in Asymptote; the creators decided to omit it because the corresponding `n--` notation would interfere with the use of the notation `p -- q` to indicate a line segment.

Here is some code demonstrating a number of useful transforms:



```
settings.outformat = "pdf";
size(3cm,0);
path p = box((0,0), (1,1));
draw(p, black + linewidth(2.0pt));
draw(shift(1,2)*p, blue);
draw(xscale(1.6)*p, green);
draw(yscale(1.4)*p, orange);
draw(scale(1.8)*p, red);
draw(rotate(60)*p, purple); /*Rotate 60
    degrees*/
```

Transforms can be composed with one another using the `*` operator. For instance, to halve the height of a path, rotate it by 45°, and translate it two to the left (in that order), you can do the following:



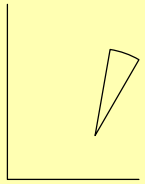
```
settings.outformat = "pdf";
size(3cm,0);
path p = unitcircle;
draw(p, black);
path q = shift(-2,0) * rotate(45) *
    yscale(0.5) * p;
draw(q, blue);
```

2.8 Arcs and margins

The function

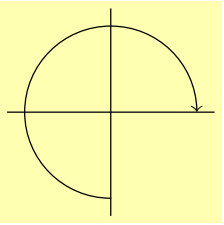
```
path arc(pair c, real r, real angle1, real angle2);
```

creates an arc centered at `c` with radius `r` from `angle1` to `angle2` specified in degrees:

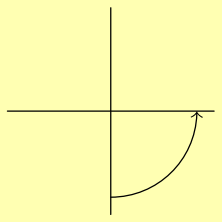


```
settings.outformat="pdf";
size(2cm,0);
draw((3,0)--(0,0)--(0,4));
draw((2,1) -- arc((2,1), 2, 60, 80) -- cycle);
```

The arc goes counterclockwise if `angle1 < angle2`, clockwise otherwise:



```
settings.outformat="pdf";
size(3cm,0);
draw((-1.2,0)--(1.2,0));
draw((0,-1.2)--(0,1.2));
/* An arc from 270 to 0 goes clockwise. */
draw(arc((0,0), r=1, angle1=270, angle2=0),
    arrow=Arrow(TeXHead));
```

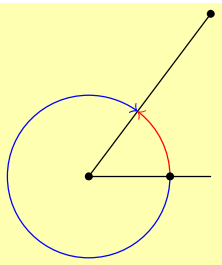


```
settings.outformat="pdf";
size(3cm,0);
draw((-1.2,0)--(1.2,0));
draw((0,-1.2)--(0,1.2));
/* An arc from -90 to 0 goes
    counterclockwise. The same effect could be
    achieved by drawing an arc from 270 to
    360. */
draw(arc((0,0), r=1, angle1=-90, angle2=0),
    arrow=Arrow(TeXHead));
```

There is another useful function for drawing arcs:

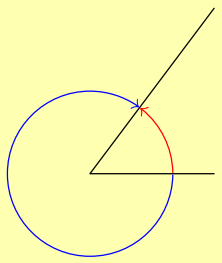
```
path arc(pair c, explicit pair z1, explicit pair z2,
    bool direction = CCW);
```

This function produces an arc centered at `c`, starting at the point `z1` and ending on the line from `c` to `z2`. The direction is specified by either `direction = CW` (clockwise) or the default `direction=CCW` (counterclockwise). This function can be quite convenient for specifying an arc from one line to another:



```
settings.outformat="pdf";
size(3cm,0);
draw((3,0) -- (0,0) -- (3,4));
draw(arc((0,0), (2,0), (3,4)),
    arrow=Arrow(TeXHead), red);
draw(arc((0,0), (2,0), (3,4), direction=CW),
    arrow=Arrow(TeXHead), blue);
dot((0,0));  dot((2,0));  dot((3,4));
```

While this looks good at first glance, Janet is distressed that upon magnification, the two arrow tips both cross into the black line rather than stopping at its edge. Here is the code to fix this using the optional parameter `margin=` for the `draw()` command:

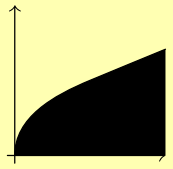


```
settings.outformat="pdf";
size(3cm,0);
draw((3,0) -- (0,0) -- (3,4));
real linewidth = linewidth(currentpen);

/* A path drawn with margin=ArrowMargins will
    be shortened at the end by 0.5 linewidth
    and at the beginning by the full
    linewidth. */
margin ArrowMargins = TrueMargin(linewidth, 0.5 linewidth);
draw(arc((0,0), (2,0), (3,4)), arrow=Arrow(TeXHead), red,
    margin=ArrowMargins);
draw(arc((0,0), (2,0), (3,4), direction=CW),
    arrow=Arrow(TeXHead), blue, margin=ArrowMargins);
```

The dots have been omitted to show the full effect, which will nevertheless be visible only on close inspection (probably with high zoom).

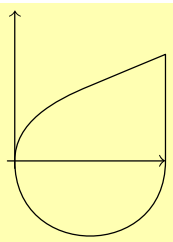
2.9 Filling a region

Next, Janet would like to fill the region under the half-parabola. This may be accomplished by creating a cyclic path and filling it with the `fill` command:



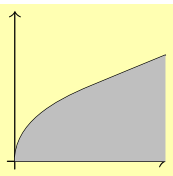
```
settings.outformat="pdf";
unitsize(1cm);
draw((-.1,0) -- (2,0), arrow=Arrow(TeXHead));
draw((0,-.1) -- (0,2), arrow = Arrow(TeXHead));
draw((0,0){up} .. (1,1) .. (2,sqrt(2)));
fill((0,0){up} .. (1,1) .. (2,sqrt(2))
      -- (2,0) -- cycle);
```

Note the use of `cycle` to close the path. Also note that the `..` and `--` operators can be combined to produce a path that is curved some places and straight others. If `.. cycle` were used instead of `--cycle`, the path would be closed smoothly:



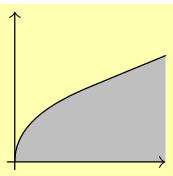
```
settings.outformat="pdf";
unitsize(1cm);
draw((-.1,0) -- (2,0), arrow=Arrow(TeXHead));
draw((0,-.1) -- (0,2), arrow = Arrow(TeXHead));
draw((0,0){up} .. (1,1) .. (2,sqrt(2))
      -- (2,0) .. cycle);
```

Returning to the originally desired picture, Janet really wants it filled with a gray color rather than black. This is not hard to achieve—she can just add an option to the `fill` command specifying what color she wants used.



```
        ⋮
fill((0,0){up} .. (1,1) .. (2,sqrt(2)) -- (2,0)
    -- cycle, mediumgray);
```

Janet almost immediately notices a problem: the filled area is covering other things, including half the arrowhead on the x-axis. This can be fixed by putting the `fill` command earlier, so that the other things get drawn on top of the filled area:

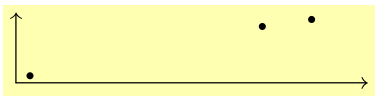


```
settings.outformat="pdf";
unitsize(1cm);
fill((0,0){up} .. (1,1) .. (2,sqrt(2))
     -- (2,0) -- cycle, mediumgray);
draw((-.1,0) -- (2,0), arrow=Arrow(TeXHead));
draw((0,-.1) -- (0,2), arrow = Arrow(TeXHead));
draw((0,0){up} .. (1,1) .. (2,sqrt(2)));
```

Janet thinks that looks much better. Vincent agrees.

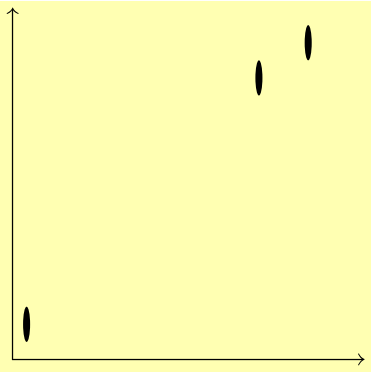
2.10 Drawing a dot at a point

One thing Janet imagines the `fill` command might be useful for is if she wants to draw a point—say, for a scatter plot:



```
settings.outformat="pdf";
size(5cm,5cm);
draw((0,0) -- (50,0), arrow=Arrow(TeXHead));
draw((0,0) -- (0,10), arrow=Arrow(TeXHead));
real r = 0.5;
fill(circle((2,1),r));
fill(circle((35,8),r));
fill(circle((42,9),r));
```

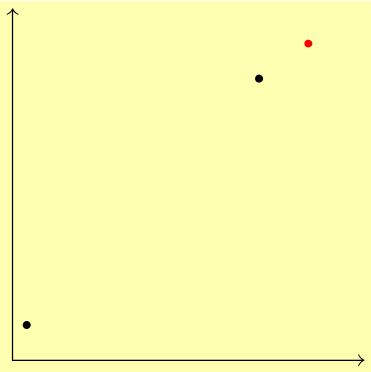
Vincent believes there may be trouble here, however, in case the plot needs to be rescaled. For data plots like this, the actual x and y scales are typically not in the same units, so there is no reason to keep the aspect ratio constant:



```
settings.outformat = "pdf";
size(5cm,5cm, keepAspect=false);
draw((0,0) -- (50,0),
    arrow=Arrow(TeXHead));
draw((0,0) -- (0,10),
    arrow=Arrow(TeXHead));
real r = 0.5;
fill(circle((2,1),r));
fill(circle((35,8),r));
fill(circle((42,9),r));
```

Unfortunately, this highlights a key weakness of drawing points by filling circles: the "points" obtained thus can change size and even shape when the picture is rescaled.

Fortunately, there is a command designed for precisely this sort of thing: the `dot` command, which draws a dot at a specified point that will remain the same size and shape even after rescaling.

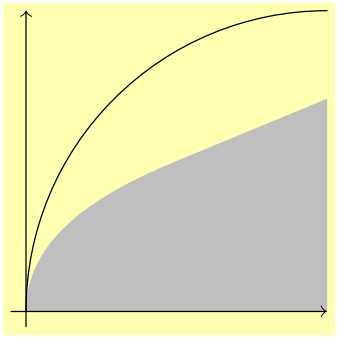


```
settings.outformat = "pdf";
size(5cm,5cm, keepAspect=false);
draw((0,0) -- (50,0),
    arrow=Arrow(TeXHead));
draw((0,0) -- (0,10),
    arrow=Arrow(TeXHead));
dot((2,1));
dot((35,8));
dot((42,9), red);
```

In the picture above, the last dot is drawn in red simply to demonstrate how to do such a thing.

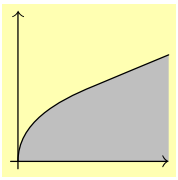
2.11 Named paths and variables

One thing that is starting to bother Vincent is that there is now a certain amount of duplicate code. If Janet were to decide that she wanted to use, say, a quarter-circle rather than a parabola, she'd have to remember to change the path in two different places, or she'd end up with something like this:



Janet thinks she can probably avoid such mistakes, but Vincent insists that even the most experienced computer programmers make mistakes like this unless they take measures to avoid redundant code. Fortunately, in Asymptote, such measures are not difficult: create a single path with a name (like `s`, for instance), and then use the name of the path rather than re-writing it entirely. Here's how this might work for the example in question:

```
settings.outformat="pdf";
unitsize(1cm);
path s = (0,0){up} .. (1,1) .. (2,sqrt(2));
fill(s -- (2,0) -- cycle, mediumgray);
draw((-.1,0) -- (2,0), arrow=Arrow(TeXHead));
draw((0,-.1) -- (0,2), arrow = Arrow(TeXHead));
draw(s);
```



Janet asks Vincent what the word "path" is doing on the third line; why would it not work just to write something like

```
s = (0,0){up} .. (1,1) .. (2,sqrt(2));
```

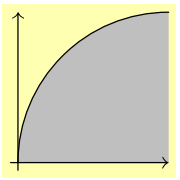
to define `s`? Vincent replies that this is a feature of C-like programming languages, including Asymptote. The letter `s` is something called a *variable*. This basically means that the programmer is allowed to assign—and later reassign—what the symbol `s` means. In this way, it is a lot like a macro in TEX or LATEX. However, unlike macros in TEX, variables in most programming languages have a specified type. Thus, for instance, the variable `s` above has type `path`. One characteristic of C-like languages is that the type of the variable must be specified the first time the variable is used. This is more or less how Asymptote knows that you are creating a new variable rather than re-assigning one that has already been created. If Janet were to omit the word `path` from the declaration

```
path s = (0,0){up} .. (1,1) .. (2,sqrt(2));
```

then Asymptote would assume she was trying to reassign an already existing `path` variable named `s`. If there were no such variable, or if the existing variable were not of type `path`, Asymptote would exit with an error message.

In any case, with the code set up this way, changing the definition of `s` will automatically change both the curve drawn and the region filled:

```
  ⋮
path s = (0,0){up} .. (2-sqrt(2), sqrt(2)) ..
    (2,2);
fill(s -- (2,0) -- cycle, mediumgray);
  ⋮
draw(s);
```

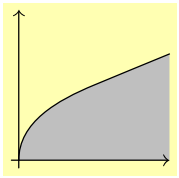


There's at least one more redundancy in the code as currently written: the arrowhead should be the same for both axes. To make sure this happens, Janet decides to create a variable called `axisarrow` and assign it the value `Arrow(TeXHead)`. Vincent tells her the type of this variable should be `arrowbar`.

Janet wonders why the type is called `arrowbar` rather than simply `arrow`. As it turns out, the creators of Asymptote decided that defining a bar on the end of a path (like the one on the left of the arrow $\mapsto$) is quite similar to defining an arrow tip. So, they combined the two into a single type and called it `arrowbar`. Janet will find herself putting both arrows and bars on the end of a line segment before the end of this tutorial.

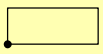
While not an issue of redundancy, the code could also be made more readable by making the ranges of the axes into named variables. The type `real`, short for "real number," is the appropriate type to use for this variable. Vincent notes that this is a departure from conventional C-like languages, which use `double`, primarily for historical reasons.

As long as Janet is deliberately making the code more readable, Vincent suggests that she should also add in some blank lines to group similar kinds of statements together. And since it does not really seem to make much difference whether the curve is drawn before or after the axes, Janet also switches the order to group related commands together. Note, however, that if she were to draw the path `s` before filling it, that *would* make a difference in the appearance of the picture. As a final note, Janet also takes advantage of the `xmax` variable in drawing and filling the path.



```
settings.outformat = "pdf";
unitsize(1cm);

real xmin = -0.1;
real xmax = 2;
real ymin = -0.1;
real ymax = 2;

path s = (0,0){up} .. (1,1) .. (xmax,sqrt(xmax));
fill(s -- (xmax,0) -- cycle, mediumgray);
draw(s);

arrowbar axisarrow = Arrow(TeXHead);
draw((xmin,0) -- (xmax,0), arrow=axisarrow);
draw((0,ymin) -- (0,ymax), arrow = axisarrow);
```

This is probably as good a place as any to mention that another important type in Asymptote is `pair`:

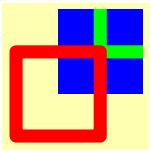


```
settings.outformat = "pdf";
size(1.5cm, 0);
pair botleft = (-1,0);
pair topright = (2.5,1.4);
draw(box(botleft, topright));
dot(botleft);
```

2.12 Clipping a picture

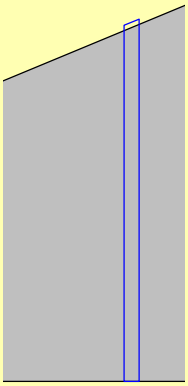
Janet will soon be doing some more detailed work on the image, which will benefit greatly from enlargement. In order to save space in this tutorial, we'll show how to clip graphics. The key is to use the `clip` command, together with a path specifying the area to be clipped. One key difference from TikZ that catches Janet somewhat by surprise is that in Asymptote, the `clip` command clips everything that came *before* it, but not what comes afterwards. (In TikZ, it's the other way around.) Here's an example, in which the blue square indicates the clipped area. The green path, which is drawn before the `clip` command, is clipped; the red path, which is drawn afterwards, is not.

```
settings.outformat="pdf";
size(2cm);
path thebox = box((0,0),(1,1));
fill(thebox, blue);
draw(shift(.5,.5)*thebox,green+linewidth(5pt));
clip(thebox);
draw(shift(-.5,-.5)*thebox,red+linewidth(5pt));
```



2.13 Path times and subpaths

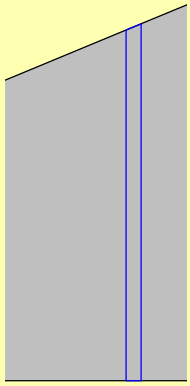
Now, Janet would like to draw the strip that will, in the 3d version, be revolved to make a disk. Here's a first attempt (at the 2d version):



```
settings.outformat="pdf";
unitsize(4cm);

real xmin = -0.1;
real xmax = 2;
real ymin = -0.1;
real ymax = 2;

path s = (0,0){up} .. (1,1) ..
    (xmax,sqrt(xmax));
fill(s -- (xmax,0) -- cycle, mediumgray);
draw(s);

arrowbar axisarrow = Arrow(TeXHead);
draw((xmin,0) -- (xmax,0), arrow=axisarrow);
draw((0,ymin) -- (0,ymax), arrow = axisarrow);

real x = 1.4;
real dx = .05;

draw((x,0) -- (x,sqrt(x)) -- (x+dx,sqrt(x+dx))
     -- (x+dx,0) --  cycle, blue);

clip(box((1,ymin),(1.6,1.5)));
```

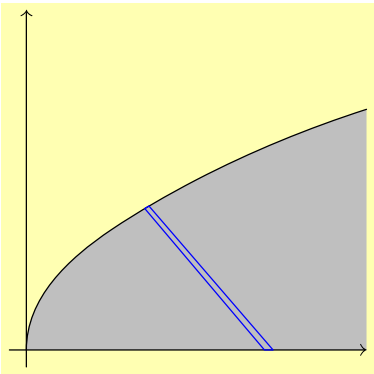
Janet immediately notices that the top of the strip is too high. She realizes that in fact it is the curve that is too low; it's supposed to resemble the graph of $\sqrt{x}$, but clearly it's a bit off. However, before figuring out how to improve the curve, Vincent wants to figure out how to make the top of the strip match the curve that was actually drawn.

A promising-looking command is `subpath(path p, real a, real b);`. According to the documentation, this "returns the subpath of `p` running from path time `a` to path time `b`". Vincent and Janet aren't sure what exactly "path time" means, but they decide to experiment rather than looking it up first:



```
settings.outformat="pdf";
unitsize(4cm);
real xmin = -0.1;
real xmax = 2;
real ymin = -0.1;
real ymax = 2;

path s = (0,0){up} .. (1,1) ..
    (xmax,sqrt(xmax));
fill(s -- (xmax,0) -- cycle, mediumgray);
draw(s);

arrowbar axisarrow = Arrow(TeXHead);
draw((xmin,0) -- (xmax,0), arrow=axisarrow);
draw((0,ymin) -- (0,ymax), arrow = axisarrow);

real x = 1.4;
real dx = .05;
path striptop = subpath(s,x,x+dx);
draw((x,0) -- striptop
     -- (x+dx,0) --  cycle, blue);

clip(box((1,ymin),(1.6,1.5)));
```

At first glance, it appears to work perfectly. Unfortunately, something dreadful happens when Janet inserts the point $(1/2, \sqrt{1/2})$ into the path `s` in an effort to make it more closely resemble the graph of $y = \sqrt{x}$:



```
settings.outformat="pdf";
size(5cm,0);

real xmin = -0.1;
real xmax = 2;
real ymin = -0.1;
real ymax = 2;

path s = (0,0){up} ..
    (1/2,sqrt(1/2)) .. (1,1) ..
    (xmax,sqrt(xmax));
fill(s -- (xmax,0) -- cycle,
    mediumgray);
draw(s);

arrowbar axisarrow = Arrow(TeXHead);
draw((xmin,0) -- (xmax,0), arrow=axisarrow);
draw((0,ymin) -- (0,ymax), arrow = axisarrow);

real x = 1.4;
real dx = .05;
path striptop = subpath(s,x,x+dx);
draw((x,0) -- striptop -- (x+dx,0) --  cycle, blue);
```

Looking through the documentation more closely, Janet and Vincent realize that "path time" is dependent on the number of "nodes" on a path, i.e., the number of points that are specified when the path is defined. Consequently, inserting an extra point to help guide the path drastically changes the path times, even though it does not actually change the shape of the path that much.

2.14 The Law of Janet

After this fiasco, Janet and Vincent formulate the following guideline, and jokingly call it the "Law of Janet":

The Law of Janet. *After a path is created, it should not be used in any way that depends on how it was created.* In particular, if one path is substituted for another with identical appearance, the rest of the code should still produce the same result.

2.15 Intersections and arrays and subpaths

At first glance, the Law of Janet appears to exclude any use of path times—which would be a pity, since a number of useful commands like `subpath` rely on them. However, after further examining the documentation, Vincent realizes that there are other commands that give the path times at which one path intersects

another. By using only path times produced by these commands, it is possible to use subpaths without violating the Law of Janet.

The particular command most useful at the moment is the function `real[] times(path p, real x)` which according to the documentation "returns all intersection times of path `p` with the vertical line through `(x,0)`."

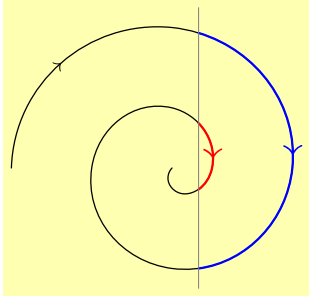
It should be noted that the return type of this function is not simply `real` (a real number), but `real[]`; the function returns an *array* of real numbers. To make sense of this, Janet gets a minimal introduction from Vincent on arrays.

Essentially, an array is an indexed list of objects of a specified type. For instance, a `real[]` (read "a real array") is an index list of `real`s. If a `real[]` is named, for instance, `a`, then the elements of this list can be accessed as `a[0]`, `a[1]`, ..., `a[n-1]`, where `n` is the length of the array. The indices themselves can be variables or expressions. For instance, if `a` is a `real[]`, the code

```
int n = a.length;
real r = a[n-1];
```

will set `r` equal to the last element of the list.

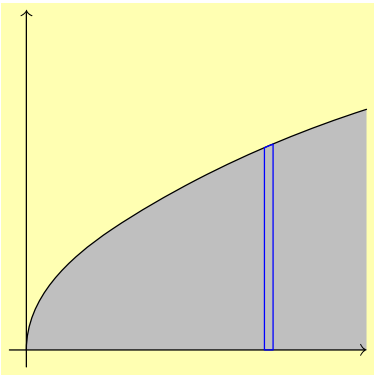
Arrays are useful when a function needs to be able to return more than one value. Since a function can return only one object, it puts all of the values into a single object—an array. In particular, the `times()` function above can return multiple different path times, which can then be plugged into the `subpath()` method. Here's an example:



```
settings.outformat="pdf";
size(4cm,0);

path p = (-2,0) .. (0,7/4) .. (6/4,0)
    .. (0,-5/4) .. (-4/4,0) .. (0,3/4)
    .. (2/4,0) .. (0,-1/4) .. (0,0);
draw(p, arrow=ArcArrow(TeXHead,
    position=0.5));

real[] isections = times(p,1/3);
draw(subpath(p,isections[0],isections[1]), blue+linewidth(0.8),
    arrow=MidArcArrow(TeXHead));
draw(subpath(p,isections[2],isections[3]), red+linewidth(0.8),
    arrow=MidArcArrow(TeXHead));
draw((1/3,-1.5) -- (1/3,2), gray + linewidth(0.2));
```

In the example above, the four points of intersection are specified by the path times `isections[0]`, `isections[1]`, `isections[2]`, and `isections[3]`; and they are arranged in the order they occur as the spiral is traced out.

For what Janet wants, there is only one point of intersection of the path with any vertical line, so what she wants is entry `[0]` of the array returned by `times(s,x)`:

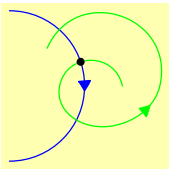


```
settings.outformat="pdf";
size(5cm,0);

real xmin = -0.1;
real xmax = 2;
real ymin = -0.1;
real ymax = 2;

path s = (0,0){up} ..
    (1/2,sqrt(1/2))  .. (1,1) ..
    (xmax,sqrt(xmax));
fill(s -- (xmax,0) -- cycle,
    mediumgray);
draw(s);

arrowbar axisarrow = Arrow(TeXHead);
draw((xmin,0) -- (xmax,0), arrow=axisarrow);
draw((0,ymin) -- (0,ymax), arrow = axisarrow);

real x = 1.4;
real dx = .05;
real t0 = times(s,x)[0];
real t1 = times(s,x+dx)[0];
path striptop = subpath(s,t0,t1);
draw((x,0) -- striptop -- (x+dx,0) --  cycle, blue);
```

There are times when it is desirable to know the intersection of a path with something more general than a vertical line. The most general version of this is to know the intersection points of two paths. The simplest function for this is

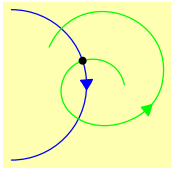
```
real[] intersect(path p, path q);
```

Assuming the two paths `p` and `q` intersect at least once, this function returns an array of length 2. Entry `[0]` gives the path time, along path `p`, of one point of intersection. Entry `[1]` gives the path time along path `q` of the same point. Thus, the two pieces of code below, which are identical except for the last line, give exactly the same result:



```
settings.outformat="pdf";
unitsize(1cm);
path p = (-1,1) .. (0,0) .. (-1,-1);
path q = (1/2,0) .. (-1/3,0) .. (1/2,-1/2) ..
    (1,0) .. (-1/2,1/2);
draw(p,blue, arrow=MidArcArrow());
draw(q,green, arrow=MidArcArrow());
```

```
real[] isections = intersect(p,q);
dot(point(p,isections[0]));
```

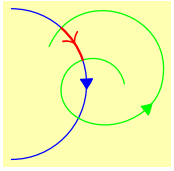


```
settings.outformat="pdf";
unitsize(1cm);
path p = (-1,1) .. (0,0) .. (-1,-1);
path q = (1/2,0) .. (-1/3,0) .. (1/2,-1/2) ..
    (1,0) .. (-1/2,1/2);
draw(p,blue, arrow=MidArcArrow());
draw(q,green, arrow=MidArcArrow());
real[] isections = intersect(p,q);
dot(point(q,isections[1]));
```

Note that the two paths shown above have more than one point of intersection. To get all of them, Janet can use the function

```
real[][] intersections(path p, path q);
```

which returns an array of `real[]`s, i.e., an array of arrays. If the `real[][]` returned is called, say, `a`, then `a[0]` is an array of two real numbers, representing the path times for `p` and for `q` of an intersection point. Significantly, when this function is used, the points are ordered according to which comes first on `p`. Thus, `subpath(p, a[0][0], a[1][0])` will be the subpath of `p` going from the first intersection point to the second intersection point:



```
settings.outformat="pdf";
unitsize(1cm);
path p = (-1,1) .. (0,0) .. (-1,-1);
path q = (1/2,0) .. (-1/3,0) .. (1/2,-1/2) ..
    (1,0) .. (-1/2,1/2);
draw(p,blue, arrow=MidArcArrow());
draw(q,green, arrow=MidArcArrow());
real[][] a = intersections(p,q);
draw(subpath(p,a[0][0], a[1][0]), red+linewidth(0.8),
    arrow=MidArrow(TeXHead));
```

If Janet were only interested in getting the intersection points, rather than path times to use for subpaths, she could use the convenience functions

```
pair intersectionpoint(path p, path q);
```

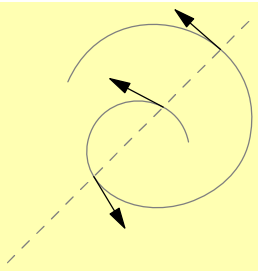
for a single point of intersection, and

```
pair[] intersectionpoints(path p, path q);
```

for a list of all intersection points (in no particular order, or at least none specified by the manual).

2.16 Tangent lines

As a calculus teacher, Janet needs a way to draw a tangent line to a curve, even though that is not needed for this particular diagram. In the past (when using TikZ), her strategy has been to specify the curve by an explicit formula, and differentiate that formula by hand to determine the slope of the tangent line. However, Asymptote has a better way: the function `dir(path p, real t)` returns the unit tangent vector to the path `p` at path time `t`. As previously discussed, path times cannot be used directly without violating the Law of Janet, but they can be constructed from intersections.

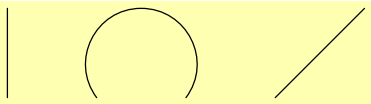


```
settings.outformat="pdf";
size(3.5cm, 0);
path p = (1/2,0) .. (-1/3,0) .. (1/2,-1/2)
    .. (1,0) .. (-1/2,1/2);
path l = (-1,-1) -- (1,1);
draw(l,dashed+gray);
draw(p, gray);
```

```
for (real[] pathtime : intersections(p,l)) {
    real t = pathtime[0];
    pair tangent = dir(p, t);
    draw(shift(point(p,t)) * scale(1/2) * ((0,0) -- tangent),
    arrow=Arrow);
}
```

2.17 Drawing disconnected paths

Technically speaking, an object of type `path` is always connected in Asymptote. Nevertheless, it is possible to form a "disconnected path"—actually an array of paths—using the "pen lift" operator `^^`:



```
settings.outformat="pdf";
size(5cm,0);
draw((0,0) -- (0,1) ^^ (1,0) ..
    (3/2,1) .. (2,0) ^^ (3,0) --
    (4,1));
```

The `draw()` command used here accepts an object of type `path[]` rather than `path`. The relevant drawing command[5] is

[5]Technically, there are two commands. The first accepts as its mandatory argument an `explicit path[]` rather than simply a `path[]`; this means that the command will *not* implicitly convert something else to a `path[]`, even if Asymptote knows how to do the conversion. A second `draw` command with mandatory argument a `guide[]` is also provided, which converts the `guide[]` to a `path[]` by an explicit conversion.

```
void draw(picture pic=currentpicture, path[] g,
          pen p=currentpen, Label legend=null,
          marker marker=nomarker);
```

The only required argument is an array of paths.

Warning: The optional arguments for the `draw(path[])` command are *not* the same as those for the more conventional `draw(path)` command. In particular, there are no arguments for adding labels, arrows, or bars.

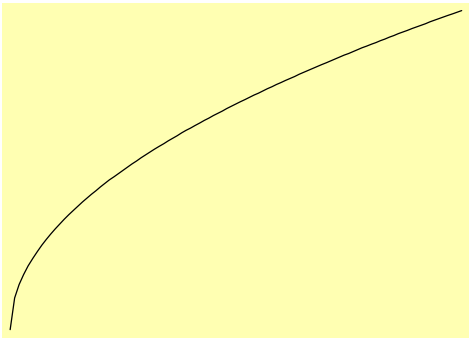
For the purposes of this tutorial, the only important optional argument is the pen `p`, which can be used to set the color and width of the paths drawn (as well as other attributes).

2.18 Graphing functions

Extrapolating a path from four points is very impressive, but Janet wonders if there is a way she could simply tell Asymptote to graph the function $y = \sqrt{x}$ for her. There is—in fact, graphing functions in a somewhat efficient manner is one of the strengths of Asymptote over a purely TeX-based system like TikZ. However, it requires that she use a piece of code that has already been written in Asymptote to do this for her. Fortunately, that is easy enough: in practice, it just means that the line `import graph;` must show up in her file before she can start making graphs. Janet thinks this is a lot like importing a LaTeX package; her husband Vincent thinks it is like using a code library. In any case, this is called importing a *module* in Asymptote.

Here's an example of how she might draw the square root function by graphing it:

```
1 settings.outformat="pdf";
2 unitsize(3cm);
3 import graph;
4 real f(real x) {
5     return sqrt(x);
6 }
7 path g = graph(f,0,2);
8 draw(g);
```

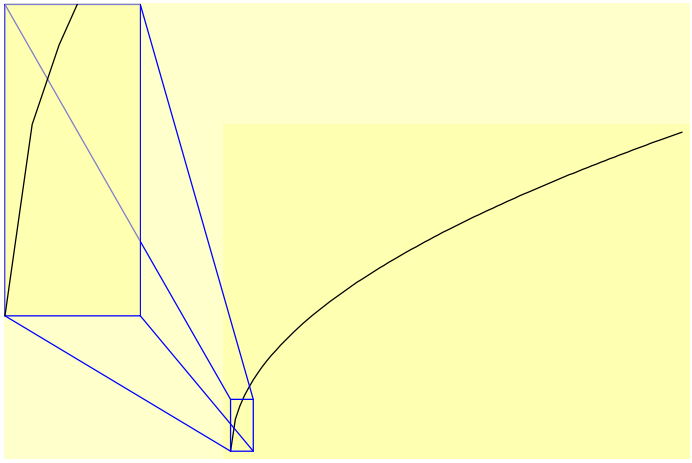


There are several features worth noticing here:

- Lines 4–6 create a function: for the rest of the code, whenever `t` is of type `real` (i.e., a real number), `f(t)` will be interpreted as the real number `sqrt(t)`.

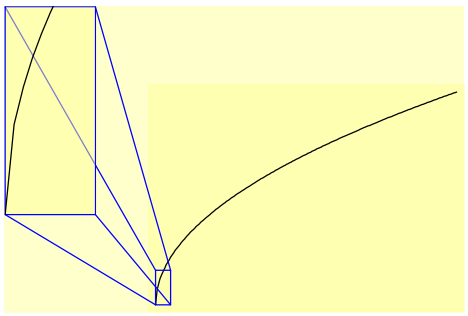
- Line 7 creates, *but does not draw*, the graph of the just-defined function $f\colon \mathbb{R} \to \mathbb{R}$ over the domain $x \in [0, 2]$. The mandatory arguments for the function `graph` are a function and the minimum and maximum x-coordinates to plot.

At first glance, the result does not look bad. However, looking more closely, Janet notices at least one corner, and it bothers her:



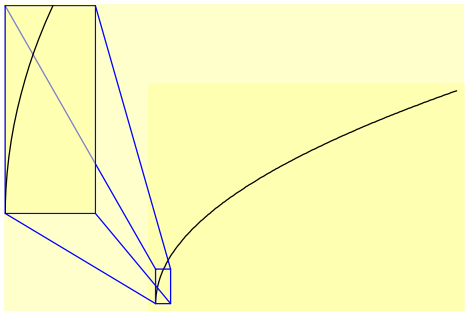
Vincent thinks it's hardly noticeable, but agrees to see what he can find in the manual. As it turns out, there are (at least) two ways to attempt to deal with this by adding optional arguments to the `graph` command. The first is simply to increase the number of points plotted, using a parameter called `n`:

```
settings.outformat="pdf";
unitsize(2cm);
import graph;
real f(real x) {
    return sqrt(x);
}
path g = graph(f,0,2,
    n=200);
draw(g);
```

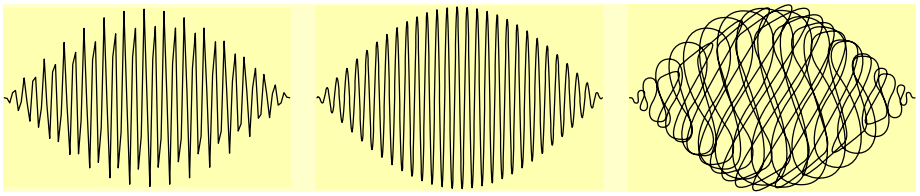


The second is to tell Asymptote to connect the points with a smooth curve rather than with line segments. This is done by adding the argument `operator ..` (The default operator, according to the manual, seems to be `operator --`.)

```
settings.outformat="pdf";
unitsize(2cm);
import graph;
real f(real x) {
    return sqrt(x);
}
path g = graph(f,0,2,
    operator ..);
draw(g);
```



In this case, the second method—telling Asymptote to draw a smooth curve—seems to work better. However, that is not always the case. Compare the following three renditions of the graph of $y = \sin x \cos 57x$:



The picture on the right, which was produced using `operator ..`, is clearly inferior to the picture in the middle, which was produced using `n=1000`. Here is the code for the three pictures:

```
settings.outformat
    = "pdf";
import graph;
unitsize(0.9cm);
real f(real x) {
    return sin(x) *
    cos(57*x); }
path g = graph(f,
    0, pi);
draw(g);
```

```
settings.outformat
    = "pdf";
import graph;
unitsize(0.9cm);
real f(real x) {
    return sin(x) *
    cos(57*x); }
path g = graph(f,
    0, pi, n=1000);
draw(g);
```

```
settings.outformat
    = "pdf";
import graph;
unitsize(0.9cm);
real f(real x) {
    return sin(x) *
    cos(57*x); }
path g =
    graph(f,0,pi,
    operator..);
draw(g);
```

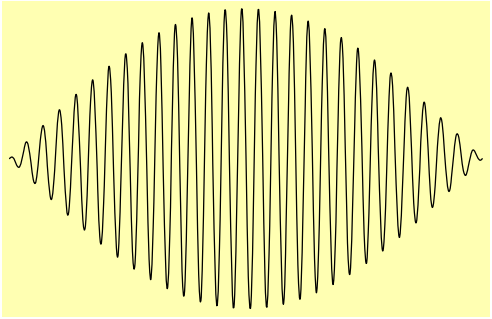
It is, of course, possible to use the option `operator ..` together with the `n=` option, but it is not always wise.

Note: `operator ..` can be problematic for graphing functions because it is designed to take full advantage of the two-dimensional drawing space, whereas functions are supposed to be somewhat limited in how they can move left and right. A better choice in this case is called `Hermite`, which creates curves without producing the awful scribble.

```
settings.outformat = "pdf";
```

```
import graph;
unitsize(2cm);
real f(real x) { return sin(x)*cos(57*x); }
path g = graph(f, 0, pi, n=200, Hermite);
draw(g);
```

The result:



2.19 Parametric graphs

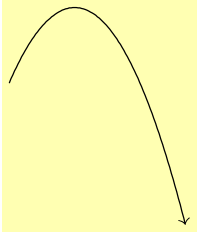
Janet's brother Jason teaches physics. In a particular example problem, a ball is thrown so that its height (in meters) after t seconds is given by

$$\begin{aligned} y(t) &= -\tfrac{1}{2}gt^2 + v_{y,0}t + y_0 \\ &= -4.5t^2 + 3.0t + 1.0 \ ; \end{aligned}$$

the horizontal distance is given by

$$\begin{aligned} x(t) &= v_x t + x_0 \\ &= 1.3t \ . \end{aligned}$$

Jason would like to be able to draw this path (for the first 0.9 seconds) without having to rewrite y in terms of x. He can do this using parametric graphs in Asymptote:



```
settings.outformat="pdf";
unitsize(2cm);
import graph;
pair F(real t) {
    return (1.3*t,  -4.5*t^2 + 3.0*t + 1.0);
}
path g = graph(F, 0, 0.9);
draw(g, arrow=Arrow(TeXHead));
```

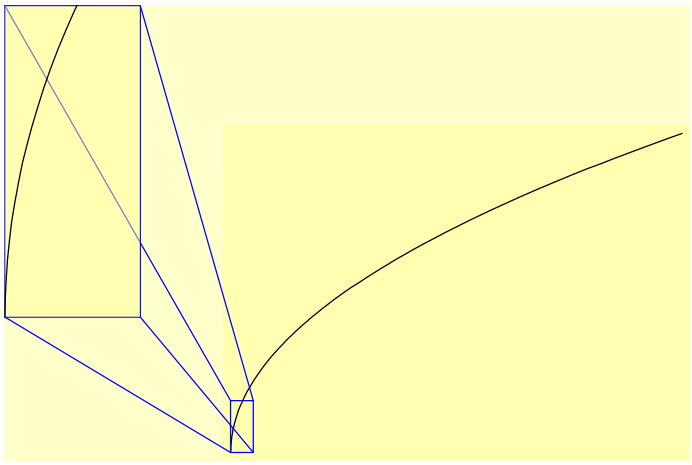
As a calculus teacher, Janet observes that the jagged corner visible in the square root graph on p. 29 is probably caused by the use of points with the x-distances evenly spaced. Since the slope of the function is very high near zero (and infinite at zero), small changes in x produce large changes in y and, consequently, long—and obtrusive—line segments. She wonders if this could be fixed by graphing x as a function of y—i.e., by graphing $x = y^2$ as y ranges from 0 to $\sqrt{2}$, rather than graphing $y = \sqrt{x}$ as x ranges from 0 to 2.

Even more generally, she would like to be able to draw a parametric graph, in which x and y are both given as functions of a third parameter t. Thus, for instance, the graph of

$$(x, y) = (t^2, t)$$

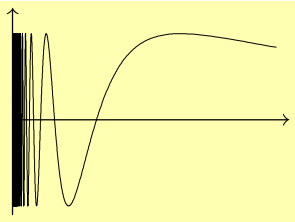
with evenly spaced choices of t might give a nicer-looking result than simply graphing $y = \sqrt{x}$. The `graph` module of Asymptote allows this more sophisticated kind of graphing as well:

```
settings.outformat="pdf";
unitsize(3cm);
import graph;
pair f(real t) { return (t^2, t); }
path g = graph(f, 0, sqrt(2));
draw(g);
```



Even though this graph is actually made up of line segments, the spacing is better so that the corners are far less noticeable.

Here's one last example that uses parametric graphing to plot the function $f(x) = \sin(1/x)$, the "topologist's sine curve," which is notoriously difficult to graph:



```
settings.outformat="pdf";
size(4cm,3cm, keepAspect=false);
import graph;
pair f(real t) {
     return (1/t, sin(t));
 }
draw(graph(f, 1, 10^4, n=5*10^5),
    thin());
draw((0,-1.1)--(0,1.3), arrow=Arrow(TeXHead));
draw((0,0)--(1.05,0), arrow=Arrow(TeXHead));
```

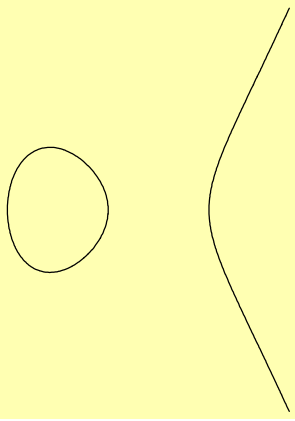
Note the use of the pen `thin()`, which shows up on a screen as being exactly one pixel thick (at every zoom level), making it possible to zoom in and see more of the oscillations before they completely obscure one another as $x \to 0$. For some reason, this particular image also seems to render much more quickly on the computer screen with the `thin()` pen than with a pen whose line width is specified; however, it may not print very nicely.

2.20 Implicitly defined curves; building arrays

Asymptote also has the capability to graph curves that are defined implicitly. This requires the module `contour` rather than `graph`. The relevant function is

```
contour(real f(real, real), pair a, pair b, real[] c);
```

it returns a `guide[][]` (i.e., an array of arrays of `guides`). If this returned `guide[][]` is saved as a variable names `thegraphs`, then for each `i`, the entry `thegraphs[i]` is a `guide[]` representing the (possibly disconnected) graph of the equation $f(x, y) = \mathtt{c}[\mathtt{i}]$. For example, the graph of $y^2 = x^3 - x$ (equivalently, $y^2 - (x^3 - x^2) = 0$) restricted to the rectangular region with corners $(-2, -2)$ and $(2, 2)$ may be obtained as follows:



```
settings.outformat="pdf";
size(4cm,0);
import contour;
real f(real x, real y) { return y^2 -
    (x^3 - x); }
guide[][] thegraphs = contour(f,
    a=(-2,-2), b=(2,2), new real[] {0});
/* The next line draws the first (and
    only) entry in thegraphs. This entry
    is itself an array, since it
    represents a disconnected path. */
draw(thegraphs[0]);
```

Warning: If you draw the paths returned by the `contour()` function, you may unwittingly assume that the drawing will automatically include the points `a` and `b`. This assumption is *false*. Drawing the paths alone will only include the smallest rectangle that contains the graphs. If you want to include the entire rectangle with corners `a` and `b`, you must do something extra, such as drawing axes or drawing the disconnected path `a ^^ b` using the pen `invisible`.

Note the use of the expression `new real[] {0}` to build an array containing exactly one entry, namely the real number `0`. This is a general technique for building arrays. For instance, the expression `new pair[] {(0,0), (1,0), (0,1), (0,1)}` produces an array with the four entries $(0,0)$, $(1,0)$, $(0,1)$, $(0,1)$. Likewise, the expression

```
(0,0)--(0,1) ^^ (1,0)..(3/2,1)..(2,0) ^^ (3,0)--(4,1)
```

is essentially[6] equivalent to the expression

```
new path[] {(0,0)--(0,1), (1,0)..(3/2,1)..(2,0), (3,0)--(4,1)}
```

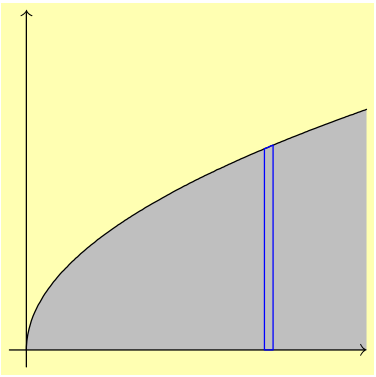
An alternative way to build an array is to declare the array and then add the items to the end one by one using the `array.push()` command. For example, the following code creates an array consisting of all nonzero integers from -5 to 15 (inclusive):

```
int[] myarray;
for (int i = -5; i <= 15; ++i) {
    if (i != 0) myarray.push(i);
}
```

2.21 The filldraw command

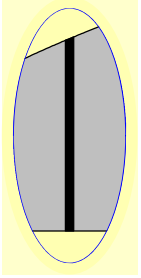
Returning to the original problem, Janet and Vincent have now progressed as far as being able to draw the following:

[6] To make it even more equivalent, change `new path[]` to `new guide[]`. The difference between paths and guides is explained in the official Asymptote documentation in the section "paths and guides" of the Programming chapter.



```
settings.outformat="pdf";
size(5cm,0);
import graph;

real xmin = -0.1;
real xmax = 2;
real ymin = -0.1;
real ymax = 2;

path s = graph(sqrt, 0, 2,
    operator..);
fill(s -- (xmax,0) -- cycle,
    mediumgray);
draw(s);
arrowbar axisarrow = Arrow(TeXHead);
draw((xmin,0) -- (xmax,0), arrow=axisarrow);
draw((0,ymin) -- (0,ymax), arrow = axisarrow);

real x = 1.4;
real dx = .05;
real t0 = times(s,x)[0];
real t1 = times(s,x+dx)[0];
path striptop = subpath(s,t0,t1);
draw((x,0) -- striptop -- (x+dx,0) --  cycle, blue);
```

What Janet actually wanted, for the blue box, was a box filled in black—and perhaps also outlined, for good measure. This can be accomplished by replacing the `draw` command in the final line of the code above with `filldraw`:



```
  ⋮

filldraw((x,0) -- striptop -- (x+dx,0) --  cycle,
    black);

  ⋮

//code to clip picture goes here
```

2.22 Adding text

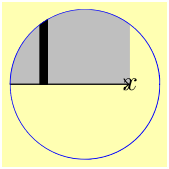
There are a number of different ways to add text to an Asymptote diagram. Probably the simplest is to use the command

```
void label(Label L, pair position);
```

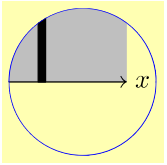
which has optional arguments

type	name	default value
`picture`	`pic`	`currentpicture`
`align`	`align`	`NoAlign`
`pen`	`p`	`currentpen`
`filltype`	`filltype`	`NoFill`

Thus, to place the text "`$x$`" at the point `(xmax,0)`, Janet can add the command `label("$x$",(xmax,0))` to the end of her Asymptote code. Here's the result after clipping:

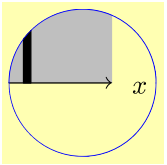


First, Janet notes that the string "`$x$`" was translated exactly into its LaTeX equivalent, which she appreciates. However, there is a more pressing issue: she did not want the text positioned exactly on top of the end of the x-axis, but to the right. For this, she can add the option "`align=E`":




```
label("$x$", (xmax,0), align=E);
```

This is much more what she had in mind. Other standard alignments include `N`, `S`, `W`, `NE`, `SE`, ... It should also be noted that these are really abbreviations for ordered pairs: `E`, for instance, really means `(0,1)`. As such, alignments can be added to each other and multiplied by real numbers:




```
label("$x$", (xmax,0), align=2.5E + S/2);
```

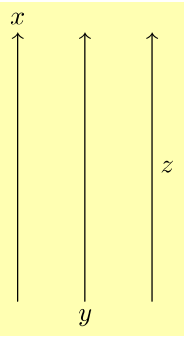
Labels can also be added directly to paths when they are drawn, using the optional parameter `Label L` of the `draw()` command. When this is done, it is typically necessary to construct a Label explicitly using the `Label()` function

```
Label Label(string s);
```

with optional parameters

type	name	default value
`string`	`size`	`""`
`position`	`position`	*no default*[7]
`align`	`align`	`NoAlign`
`pen`	`p`	`nullpen`
`embed`	`embed`	`Rotate`
`filltype`	`filltype`	`NoFill`

The most important optional parameter here is `position=`. Typical values include `position=EndPoint`, `position=MidPoint`, or `position=BeginPoint`. More generally, `position=Relative(r)` will take the real number `r` $\in [0,1]$ to specify the location of the Label as a fraction of the total arclength of the path. For instance, `MidPoint` is defined to be `Relative(0.5)`.




```
settings.outformat = "pdf";
size(2.5cm, 0);
Label Lx= Label("$x$", position=EndPoint);
Label Ly = Label("$y$", position=BeginPoint);
Label Lz = Label("$z$", position=MidPoint);
draw((0,0) -- (0,4), arrow=Arrow(TeXHead),
    L=Lx);
draw((1,0) -- (1,4), arrow=Arrow(TeXHead),
    L=Ly);
draw((2,0) -- (2,4), arrow=Arrow(TeXHead),
    L=Lz);
```

The default alignments are reasonably sensible. However, they can be changed by using the optional paramater `align` in creating the label with `Label`. For instance, the lower of the following two images is more like the one Janet is going to want in her diagram:

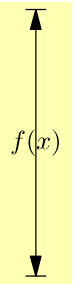

```
settings.outformat="pdf";
unitsize(0.3cm);
Label L = Label("$dx$", position=MidPoint);
draw((0,0) -- (1,0), L=L, bar=Bars);
```


```
settings.outformat="pdf";
unitsize(0.3cm);
Label L = Label("$dx$", position=MidPoint, align=2N);
draw((0,0) -- (1,0), L=L, bar=Bars);
```

[7] An "optional parameter with no default" is obtained by *overloading* the function, i.e., by defining multiple functions with the same name but different arguments.

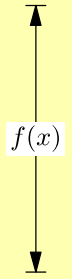
Note the use of the option `bar=Bars` on the draw command to produce bars perpendicular to the ends of the path. The other options for this parameter are `None` (the default), `BeginBar`, `EndBar`, and `Bar` (which is the same as `EndBar`).

Another label that Janet wants to add to her diagram is one indicating the height of the $dx \times f(x)$ strip. For this, she wants a vertical line with arrows and bars on both ends, and with the label $f(x)$ right on top of the midpoint. For the latter effect, the option `align=(0,0)` in the construction of the label is called for:




```
settings.outformat="pdf";
unitsize(3cm);
defaultpen(fontsize(10pt));
Label L = Label("$f(x)$", align=(0,0),
    position=MidPoint);
draw((0,0) -- (0,sqrt(1.4)), L=L, arrow=Arrows(),
    bar=Bars);
```

There is, unfortunately, one more problem: the part of the line behind the label $f(x)$ needs to be hidden for readability. Or, to put it another way, a box around the label needs to be filled, with the same color as the background, to hide the line behind it. Janet can accomplish this by using the option `filltype=Fill(white)` in the construction of the label, where `white` should be replaced by whatever the background color really is.



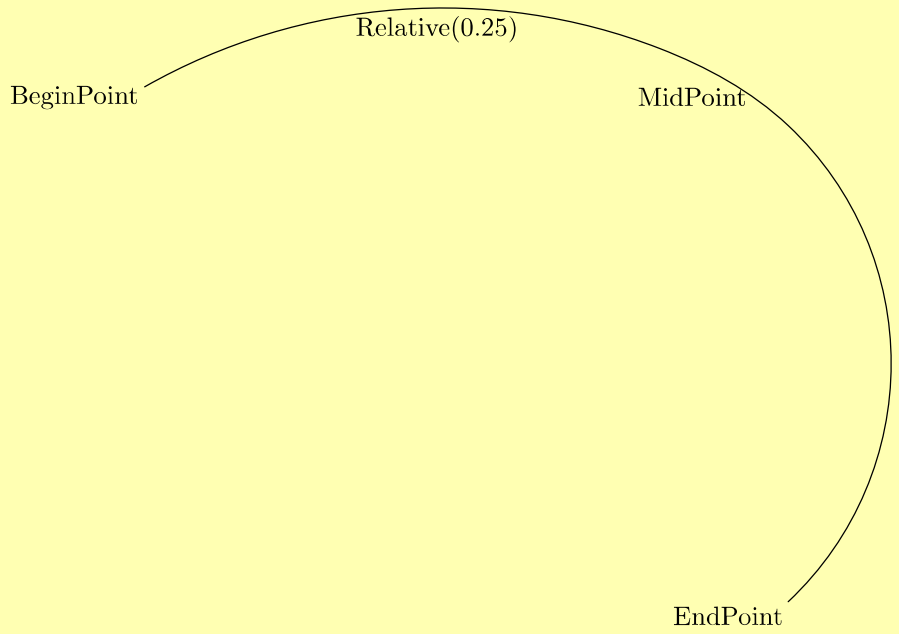

```
settings.outformat="pdf";
unitsize(3cm);
defaultpen(fontsize(10pt));
Label L = Label("$f(x)$", align=(0,0),
    position=MidPoint, filltype=Fill(white));
draw((0,0) -- (0,sqrt(1.4)), L=L, arrow=Arrows(),
    bar=Bars);
```

2.23 Adding multiple labels to a single path

To add multiple labels to the same path, construct the labels separately using `Label` and then apply the method `label(Label L, path g)`:

```
settings.outformat="pdf";
defaultpen(fontsize(10pt));
size(12cm, 0);
path p =(0,0) .. (4,.3) .. (5,-.3) .. (5,-4);
draw(p);
Label L1 = Label("BeginPoint", position=BeginPoint);
```

```
label(L1, p);
Label L2 = Label("MidPoint", position=MidPoint);
label(L2, p);
Label L3 = Label("EndPoint", position=EndPoint);
Label L4 = Label("Relative(0.25)", position=Relative(0.25));
label(L3, p);
label(L4, p);
```

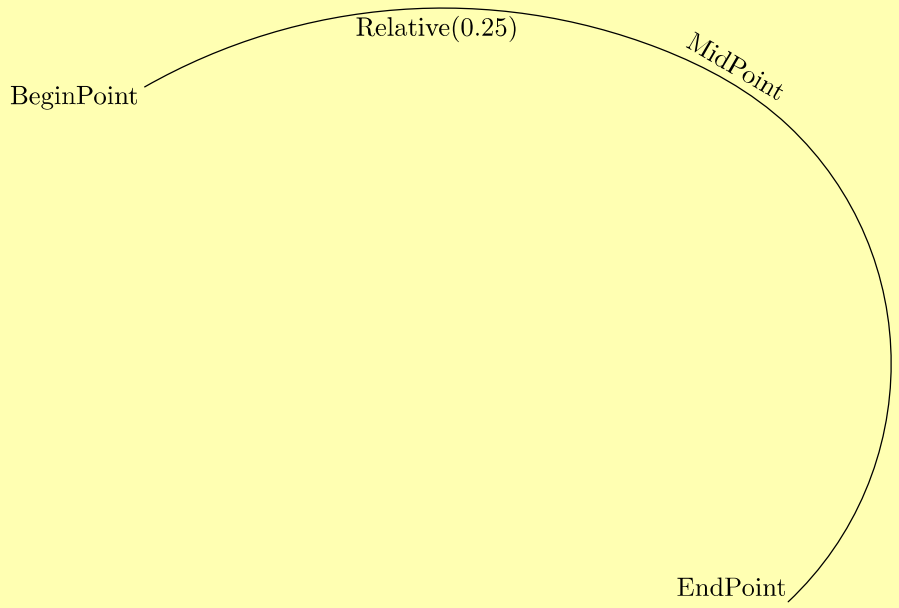


Janet thinks it is unfortunate that it takes two[8] lines of code to add each label to the path, and also that there is no obvious way to make the text of the label slope along the path. After consulting the source code in `plain_Label.asy`, she and Vincent manage to cobble together a new command, which they call `pathlabel`, that seems to alleviate both these issues:

```
void pathlabel(picture pic = currentpicture, Label L, path g,
    real position=0.5, align align=NoAlign, bool sloped=false,
    pen p=currentpen, filltype filltype=NoFill) {
  Label L2 = Label(L, align, p, filltype,
    position=Relative(position));
  if (sloped) {
    pair direction = dir(g, reltime(g, position));
    real angle = degrees(atan2(direction.y, direction.x));
    L2 = rotate(angle)*L2;
```

[8]This can be abbreviated to a single arguably more confusing line of code by defining the `Label` within the `label` command, e.g., `label(Label("MidPoint",position=MidPoint),p);`

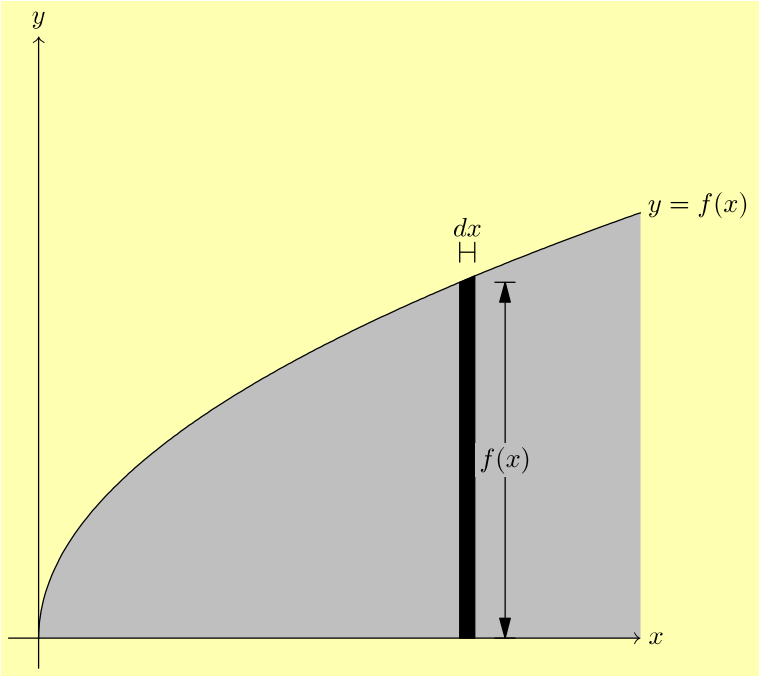
```
  }
  label(pic, L2, g);
}

size(12cm, 0);
defaultpen(fontsize(10pt));
settings.outformat="pdf";
path p =(0,0) .. (4,.3) .. (5,-.3) .. (5,-4);
draw(p);
pathlabel("BeginPoint", p, position=0);
pathlabel("Relative(0.25)", p, position=0.25);
pathlabel("MidPoint", p, position=0.5, align=Relative(W),
    sloped=true);
pathlabel("EndPoint", p, align=Relative(E), position=1.0);
```



Note also the use of the option `align=Relative(pair)`, which tells Asymptote to align the label relative to the path, with "north" pointing in the tangent direction to the path. In particular, `align=Relative(E)` will put the label to the right of the path direction (i.e., below the path, if it is going from left to right), while `align=Relative(W)` will put the label to the left of the path direction (above the path, if it is going from left to right).

2.24 Drawing objects of a fixed (unscalable) size

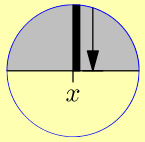
If we combine all that we have done so far toward the desired image, we get the following:



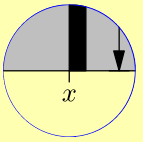

```
settings.outformat="pdf";
unitsize(4cm);
defaultpen(fontsize(10pt));
import graph;

real xmin = -0.1;
real xmax = 2;
real ymin = -0.1;
real ymax = 2;

real f(real x) { return sqrt(x); }
path s = graph(f, 0, 2, operator..);
pen fillpen = mediumgray;
fill(s -- (xmax,0) -- cycle, fillpen);
draw(s, L=Label("$y=f(x)$", position=EndPoint));

real x = 1.4;
real dx = .05;
real t0 = times(s,x)[0];
real t1 = times(s,x+dx)[0];
path striptop = subpath(s,t0,t1);
filldraw((x,0) -- striptop -- (x+dx,0) --  cycle, black);
```

```
real barheight = f(x) + .1;
Label dxlabel = Label("$dx$", position=MidPoint, align=2N);
draw((x,barheight) -- (x+dx, barheight), L=dxlabel, bar=Bars);

real barx = x + dx + 0.1;
Label fxlabel = Label("$f(x)$", align=(0,0), position=MidPoint,
    filltype=Fill(fillpen));
draw((barx,0) -- (barx, f(x)), L=fxlabel, arrow=Arrows(),
    bar=Bars);

arrowbar axisarrow = Arrow(TeXHead);
Label xlabel = Label("$x$", position=EndPoint);
draw((xmin,0) -- (xmax,0), arrow=axisarrow, L=xlabel);
Label ylabel = Label("$y$", position=EndPoint);
draw((0,ymin) -- (0,ymax), arrow = axisarrow, L=ylabel);
```

This is getting close to the desired result, but there are a few missing bits. First of all, Janet wants a tick mark on x-axis which labels the horizontal position of the strip as x. Moreover, this tick should remain the same size—say, 1.5 millimeters long—even if the image is rescaled. Fortunately, there is a variant of the `draw` command that draws a scale-proof object at a scalable location (like $(x, 0)$, which will change if the image is rescaled). This variant is called when the first parameter passed to the `draw` command is an ordered pair. Note that in the two examples below, the tick mark is the same size even though the scaling is different:

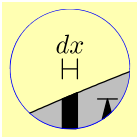


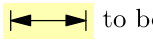
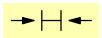

```
size(2cm);

    ⋮

path tick = (0,0) --
    (0,-0.15cm);
Label ticklabel =
    Label("$x$",
    position=EndPoint);
draw((x,0), tick,
    L=ticklabel);
path clippath=circle((x,0),
    0.5);
draw(clippath, blue);
clip(clippath);
```



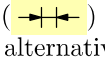

```
size(2cm);

    ⋮

path tick = (0,0) --
    (0,-0.15cm);
Label ticklabel =
    Label("$x$",
    position=EndPoint);
draw((x,0), tick,
    L=ticklabel);
path clippath=circle((x,0),
    0.2);
draw(clippath, blue);
clip(clippath);
```

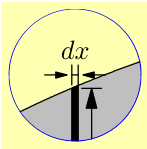
Another detail Janet finds unsatisfying is the way that the dx length is indicated:



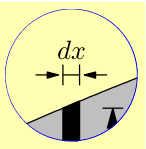
These bars are clearly too close together for a construction like  to be reasonable. To have this look "nicer," Janet would like to imitate a style she saw in a textbook in which arrows pointing in from the either side:  . Furthermore, the lengths of these arrows should be given in absolute measurements that do not change with scaling; say, each arrow should be 3 millimeters long. This could be done using the `draw` command discussed above. However, for something like this, it is easier to use the `arrow` command: `arrow(pair b, pair dir, real length=arrowlength)` will draw an arrow pointing to `b` from direction `dir`, with length `length` specified in absolute (unscalable) units.


```
settings.outformat="pdf";
size(1cm,0);
draw((0,0) -- (1,0), bar=Bars);
arrow((0,0), W, length=0.3cm);
arrow((1,0), E, length=0.3cm);
```

Unfortunately, Janet thinks that there is a bit too much space at the arrow tips. Vincent consults the documentation and realizes that this is a consequence of the default option `margin=EndMargin`, which is designed for an arrow pointing to a label. Predefined alternatives include using `arrow(..., margin=NoMargin);` () and `arrow(..., margin=DotMargin);` ( ). Of these, the best alternative for this situation seems to be `margin=DotMargin`, which was designed for an arrow pointing to a dot created with the `dot` command. As desired, the resulting arrows remain the same length when the picture is scaled differently:



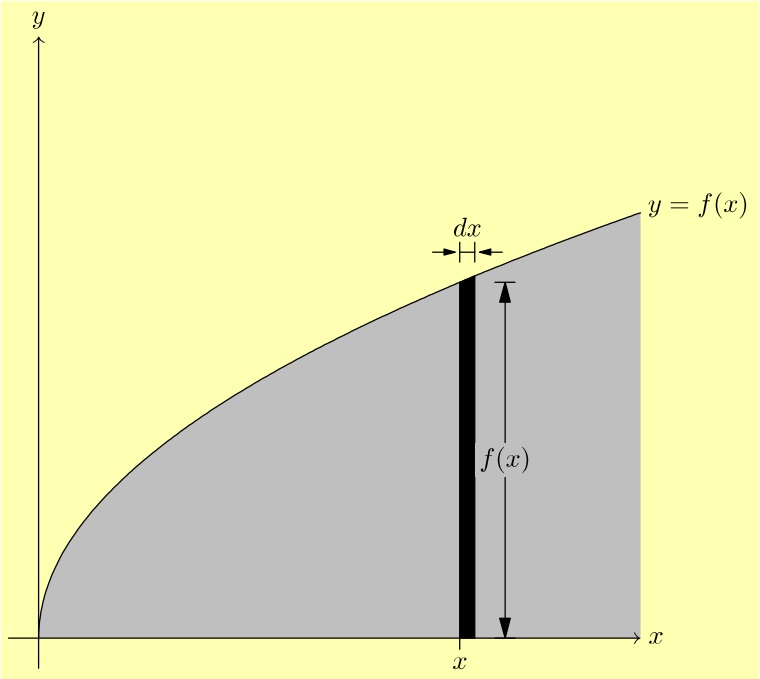

```
size(2cm);

    ⋮

real myarrowlength = 0.3cm;
margin arrowmargin =
    DotMargin;
arrow((x,barheight), W,
    length=myarrowlength,
    margin=arrowmargin);
arrow((x+dx,barheight), E,
    length=myarrowlength,
    margin=arrowmargin);

path clippath = circle(
    (x+dx/2,barheight), 0.5);
draw(clippath, blue);
clip(clippath);
```



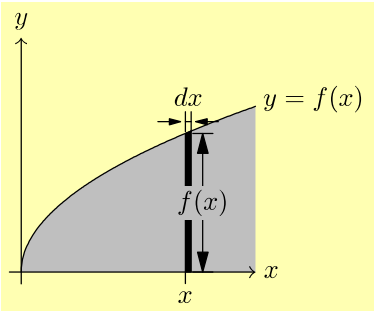

```
size(2cm);

    ⋮

real myarrowlength = 0.3cm;
margin arrowmargin =
    DotMargin;
arrow((x,barheight), W,
    length=myarrowlength,
    margin=arrowmargin);
arrow((x+dx,barheight), E,
    length=myarrowlength,
    margin=arrowmargin);

path clippath = circle(
    (x+dx/2,barheight), 0.2);
draw(clippath, blue);
clip(clippath);
```

2.25 Drawing objects shifted by an unscalable amount

Let's review the full, unclipped version of what Janet has accomplished so far:



Janet at first thinks that the two-dimensional drawing is now finished. However, she believes she will need to make the scale a bit smaller for the final drawing, so she decides to try a different size just to make sure the picture scales correctly:




```
settings.outformat = "pdf";
size(5cm, 0);

    ⋮
```

While the result is not awful, there are a couple of issues. Most obviously, the label $f(x)$ goes over the bar. Less problematic, but still irritating to Janet, is the way the arrow to the lower right of the dx label goes into the shaded region, rather than remaining strictly above it.

Vincent points out that in both cases, the problem was that specific distances were scaled that should not have been: the dx label should have remained the same height above the black strip no matter how the scaling changed, and the $f(x)$ label should have remained the same distance to the right. Unfortunately, after extensively consulting the documentation, neither Janet nor Vincent can find a

completely satisfactory way to include an unscalable length in the positioning of an object.

Finally, after weeks of considering the problem on-and-off and even consulting the source code for the basic Asymptote modules, Vincent manages to create a drawing command that does what is required. First, he adds the following definition[9] to the beginning of the file:

```
void drawshifted(path g, pair trueshift, picture pic =
    currentpicture, Label label="", pen pen=currentpen,
    arrowbar arrow=None, arrowbar bar=None, margin
    margin=NoMargin, marker marker=nomarker)
{
  pic.add(new void(frame f, transform t) {
      picture opic;
      draw(opic, L=label, shift(trueshift)*t*g, p=pen,
    arrow=arrow, bar=bar,
           margin=margin, marker=marker);
      add(f,opic.fit());
    });
  pic.addBox(min(g), max(g), trueshift+min(pen),
    trueshift+max(pen));
}
```

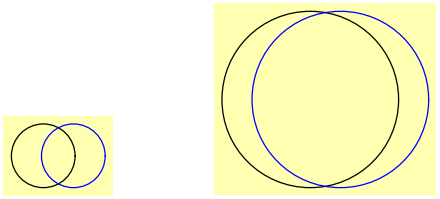
With this definition in place, the command, for instance, `drawshifted(g, (0.4cm, 0))` should draw the path `g` shifted exactly 3 millimeters to the right of its "natural" position; and this shift of 4 millimeters will remain the same, regardless of the scaling. Consider, for instance, the following code:

```
1  void drawshifted(path g, pair trueshift, picture pic =
       currentpicture, Label label="", pen pen=currentpen,
       arrowbar arrow=None, arrowbar bar=None, margin
       margin=NoMargin, marker marker=nomarker)
2  {
3    pic.add(new void(frame f, transform t) {
4        picture opic;
5        draw(opic, L=label, shift(trueshift)*t*g, p=pen,
       arrow=arrow, bar=bar,
6              margin=margin, marker=marker);
7        add(f,opic.fit());
8      });
9    pic.addBox(min(g), max(g), trueshift+min(pen),
       trueshift+max(pen));
10 }
```

[9] The original definition here was erroneous; the error was pointed out by user `mauvia` in the discussion thread at `https://sourceforge.net/p/asymptote/discussion/409349/thread/d0bc619b/?limit=25`, who also provided the core of the partial correction.

```
12 settings.outformat="pdf";
13 size(***);
14 draw(unitcircle);
15 drawshifted(unitcircle, trueshift=(0.4cm,0), pen=blue);
```

No matter how the scaling is set by changing the measurement in the `size` command (replacing the `***` on line 13), the blue circle will always be exactly four millimeters to the right of the black circle:



Warning: If the `drawshifted()` method above is used with fixed-size elements such as a label, arrow, and/or bar, then any subsequent occurrences of the functions `min(currentpicture)` and `max(currentpicture)` will ignore these fixed-size elements and may consequently be incorrect.

Recall that the vertical line with the $f(x)$ label was created with the following code:

```
real barx = x + dx + 0.1;
Label fxlabel = Label("$f(x)$", align=(0,0), position=MidPoint,
    filltype=Fill(fillpen));
draw((barx,0) -- (barx, f(x)), L=fxlabel, arrow=Arrows(),
    bar=Bars);
```

The vertical strip has coordinate x on the left and $x + dx$ on the right; in this code, the label is shifted right by a distance of 0.1 scalable units. To use instead an unscalable shift of 4.2 millimeters, Janet replaces the code above with the following:

```
/* Note: the following code will only work after the
    drawshifted() command has been defined as above. */
real barx = x + dx;
pair barshiftx = (0.42cm, 0);
Label fxlabel = Label("$f(x)$", align=(0,0), position=MidPoint,
    filltype=Fill(fillpen));
drawshifted((barx,0) -- (barx, f(x)), trueshift=barshiftx,
    label=fxlabel, arrow=Arrows(), bar=Bars);
```

The dx label  is a bit trickier, because it includes both a scalable part (the line with bars on both ends) and an unscalable part (the arrows pointing in). Janet's solution is to use the `drawshifted()` command for the scalable part, and the `draw()` variant described earlier (p. 42) to draw the arrows. See lines 40–52 of the code on page 49.

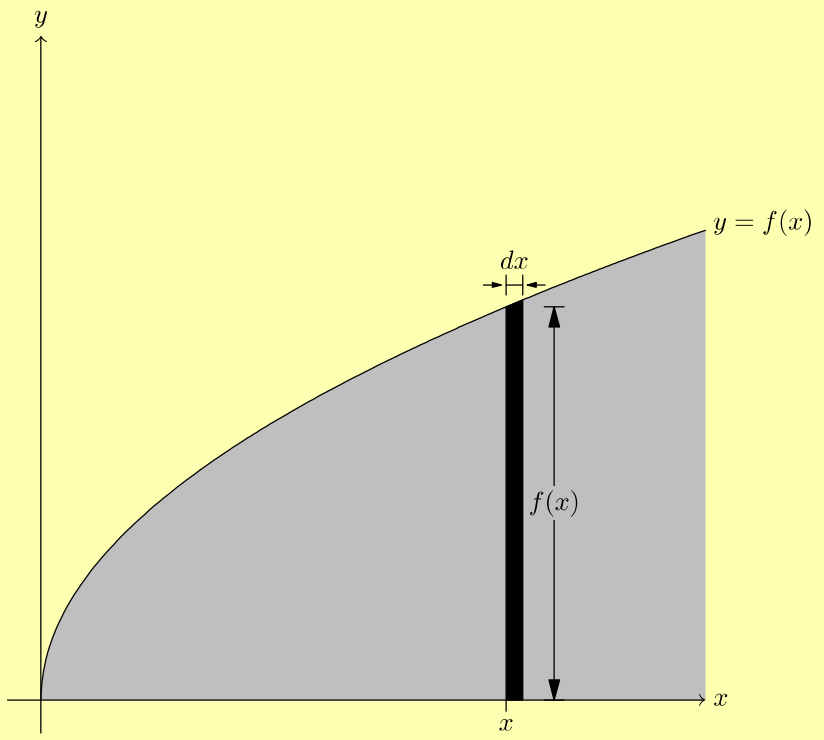
2.26 The two-dimensional picture: final result

The code:

```
//Basic settings
settings.outformat="pdf";
size(11cm, 0);
defaultpen(fontsize(10pt));
import graph;

//Define the command drawshifted, to be used later
void drawshifted(path g, pair trueshift, picture pic =
    currentpicture, Label label="", pen pen=currentpen,
    arrowbar arrow=None, arrowbar bar=None, margin
    margin=NoMargin, marker marker=nomarker)
{
  pic.add(new void(frame f, transform t) {
      picture opic;
      draw(opic, L=label, shift(trueshift)*t*g, p=pen,
    arrow=arrow, bar=bar,
            margin=margin, marker=marker);
      add(f,opic.fit());
    });
  pic.addBox(min(g), max(g), trueshift+min(pen),
    trueshift+max(pen));
}

//Save some important numbers.
real xmin = -0.1;
real xmax = 2;
real ymin = -0.1;
real ymax = 2;

//Draw the graph and fill the area under it.
real f(real x) { return sqrt(x); }
path s = graph(f, 0, 2, operator..);
pen fillpen = mediumgray;
fill(s -- (xmax,0) -- cycle, fillpen);
draw(s, L=Label("$y=f(x)$", position=EndPoint));

//Fill the strip of width dx
real x = 1.4;
real dx = .05;
real t0 = times(s,x)[0];
real t1 = times(s,x+dx)[0];
path striptop = subpath(s,t0,t1);
filldraw((x,0) -- striptop -- (x+dx,0) --  cycle, black);
```

```
40 //Draw the bars labeling the width dx
41 real barheight = f(x+dx);
42 pair barshifty = (0, 0.2cm);
43 Label dxlabel = Label("$dx$", position=MidPoint, align=2N);
44 drawshifted((x,barheight) -- (x+dx, barheight),
       trueshift=barshifty, label=dxlabel, bar=Bars);

46 //Draw the arrows pointing inward toward the dx label
47 real myarrowlength = 0.3cm;
48 margin arrowmargin = DotMargin;
49 path leftarrow = shift(barshifty) * ((-myarrowlength, 0) --
       (0,0));
50 path rightarrow = shift(barshifty) * ((myarrowlength, 0) --
       (0,0));
51 draw((x, barheight), leftarrow, arrow=Arrow(),
       margin=arrowmargin);
52 draw((x+dx, barheight), rightarrow, arrow=Arrow(),
       margin=arrowmargin);

54 //Draw the bar labeling the height f(x)
55 real barx = x + dx;
56 pair barshiftx = (0.42cm, 0);
57 Label fxlabel = Label("$f(x)$", align=(0,0), position=MidPoint,
       filltype=Fill(fillpen));
58 drawshifted((barx,0) -- (barx, f(x)), trueshift=barshiftx,
       label=fxlabel, arrow=Arrows(), bar=Bars);

60 //Draw the axes on top of everything that has gone before
61 arrowbar axisarrow = Arrow(TeXHead);
62 Label xlabel = Label("$x$", position=EndPoint);
63 draw((xmin,0) -- (xmax,0), arrow=axisarrow, L=xlabel);
64 Label ylabel = Label("$y$", position=EndPoint);
65 draw((0,ymin) -- (0,ymax), arrow = axisarrow, L=ylabel);

67 //Draw the tick mark on the x-axis
68 path tick = (0,0) -- (0,-0.15cm);
69 Label ticklabel = Label("$x$", position=EndPoint);
70 draw((x,0), tick, L=ticklabel);
```

The result:



Here's a rescaled version, obtained by substituting `size(7cm,0)` for line 3, to show that rescaling actually works properly: